#Class vs Struct in C

Feature	Class	Struct
Type	Reference type	Value type
Memory	Reference to an object	Stores the value directly
Null	Can be `null`	Cannot be `null` unless nullable
Inheritance	Supports inheritance	Does not support class inheritance
Interfaces	Can implement interfaces	Can implement interfaces
Constructor	Can have multiple constructors	Has restrictions depending on C# version
Destructor	Can have a finalizer	Cannot have a finalizer
Best used for	Complex objects and entities	Small, lightweight data values

Main Difference

.**Class:** Reference type → assignment copies the reference

.**Struct:** Value type → assignment copies the value

Relationships Between Classes

Inheritance .1

- .Represents an **IS-A** relationship
- .A derived class inherits members from a base class
- .Supports code reuse and polymorphism

Association .2

- .Represents a general relationship between two classes
- Indicates that objects of one class are connected to or interact with objects of another .class

Aggregation .3

- .Represents a **weak HAS-A** relationship

- .One class contains or groups objects of another class
- .The contained objects can exist independently

.4 Composition

- .Represents a **strong HAS-A** relationship
- .One class owns the objects of another class
- .The contained objects generally depend on the owner for their lifecycle

.5 Dependency

- .Represents a **USES** relationship
- .One class depends on another to perform a particular operation
- .Usually temporary rather than a permanent relationship

Quick Memorization

Inheritance → IS-A
Association → Related to
Aggregation → HAS-A (weak)
Composition → HAS-A (strong)
Dependency → USES